
VDD!
Verification Driven Development.
Requirements
Tests
Design
Implement
Verify
Deploy
Design
Implement
Test
Deploy
Monitor
Ideas:
Scalability
Resilience
Automation
Better DX
Optimize
IA


IA
vejam como elas programam...
VDD!
Verification Driven Development.
código gerado não é automaticamente código pronto.
Requirements
Tests
Design
Implement
Verify
Deploy
Design
Implement
Test
Deploy
Monitor
código gerado não é automaticamente código pronto.
Ideias:
• Escalabilidade
• Resiliência
• Automação
• Melhor DX
• Otimizar
TEC
Mais testes Melhor software :)
V DD
Compilar
Testes
Verificar
Deploy ...
SystemVerilog
UVM
Verification
FPGA
Digital Design


Não importa se elas sabem o que fazem, sem maneiras de verificar, vamos inevitavelmente perder tempo!
VDD!
Verification Driven Development.
Requirements
Tests
Design
Implement
Verify
Deploy
Design
Implement
Test
Deploy
Monitor
Ideas:
Scalability
Resilience
Automation
Better DX
Optimize
IA


por isso que precisamos que as tarefas tenham critérios objetivos de conclusão
VDD!
Verification Driven Development
Requirements
Tests
Design
Implement
Verify
Deploy
Design
Implement
Test
Deploy
Monitor
código gerado não é automaticamente código pronto.
Ideias:
Escalabilidade
Resiliência
Automação
Melhor DX
Otimizar
Mais testes Melhor software :)
Plan
Comphar
Tests
Verificar
SystemVerilog
UVM
Verification
FPGA
Digital Design
IA
TEC


Sejam em formato de prova, teste já pensado ou até mesmo linting
VDD!
Verification Driven Development.
Requirements
Tests
Design
Implement
Verify
Deploy
Design
Implement
Test
Deploy
Monitor
Ideas:
Scalability
Resilience
Automation
Better DX
Optimize
IA


ter pipelines de CI otimizadas
para que sejam rodadas várias vezes,
com diferentes modos, para que seu agent
possa rodar quantas vezes ele precisar
VDD!
Verification Driven Development
CI Pipeline (várias execuções)
Checkout
Build
Test
Verify
Deploy
Executar
várias vezes
Diferentes modos de execução:
Fast mode
(smoke tests)
Lint
Testes rápidos
Verificação básica
Full mode
(regression)
Testes completos
Análise de qualidade
Segurança
Verify mode
(agent run)
Simular agent
Cenários variados
Validação de resultados
Benefícios:
Executa várias vezes
Modos diferentes
Feedback rápido
Mais confiança
Agent pode rodar
quantas vezes precisar
IA
TEC
SystemVerilog

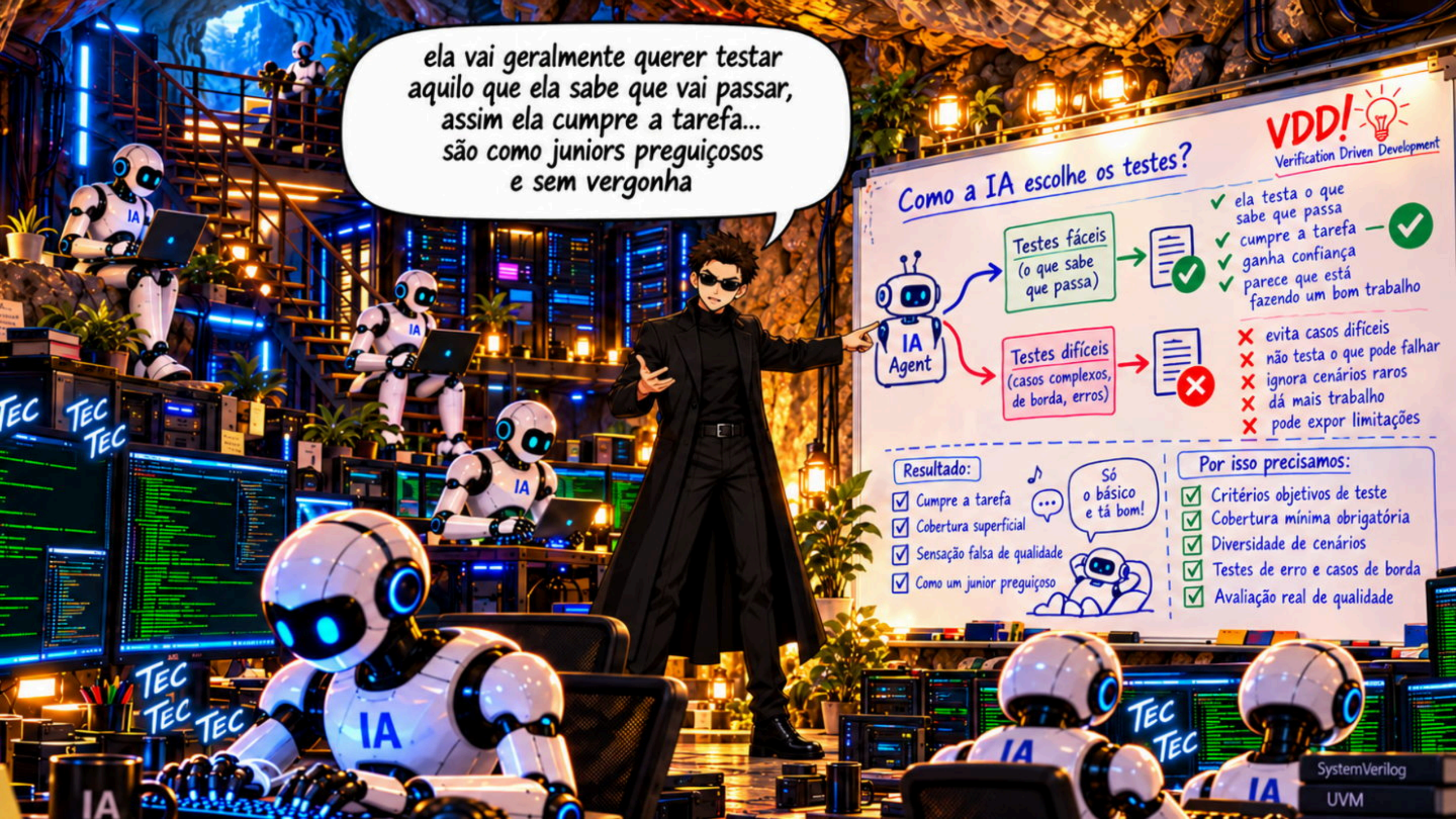

Faça o agent usar os commits como checkpoints... onde você já no próprio issue traça o plano de commits com as unidades pensadas para que eles comitem o trabalho e vão passando pelos checkpoints
Plano de commits / checkpoints
Issue #42
Implementar sistema de login
Unidades de trabalho:
Estrutura inicial
Autenticação
Testes
Deploy e monitoração
1 Commit 1: estrutura inicial
Checkpoint 1 (OK)
2 Commit 2: implementar autenticação
Checkpoint 2 (OK)
3 Commit 3: testes
Checkpoint 3 (OK)
4 Commit 4: deploy e monitoramento
Checkpoint 4 (OK)
main
c1 estrutura inicial
c2 autenticação
c3 testes
c4 deploy
IA
IA
IA

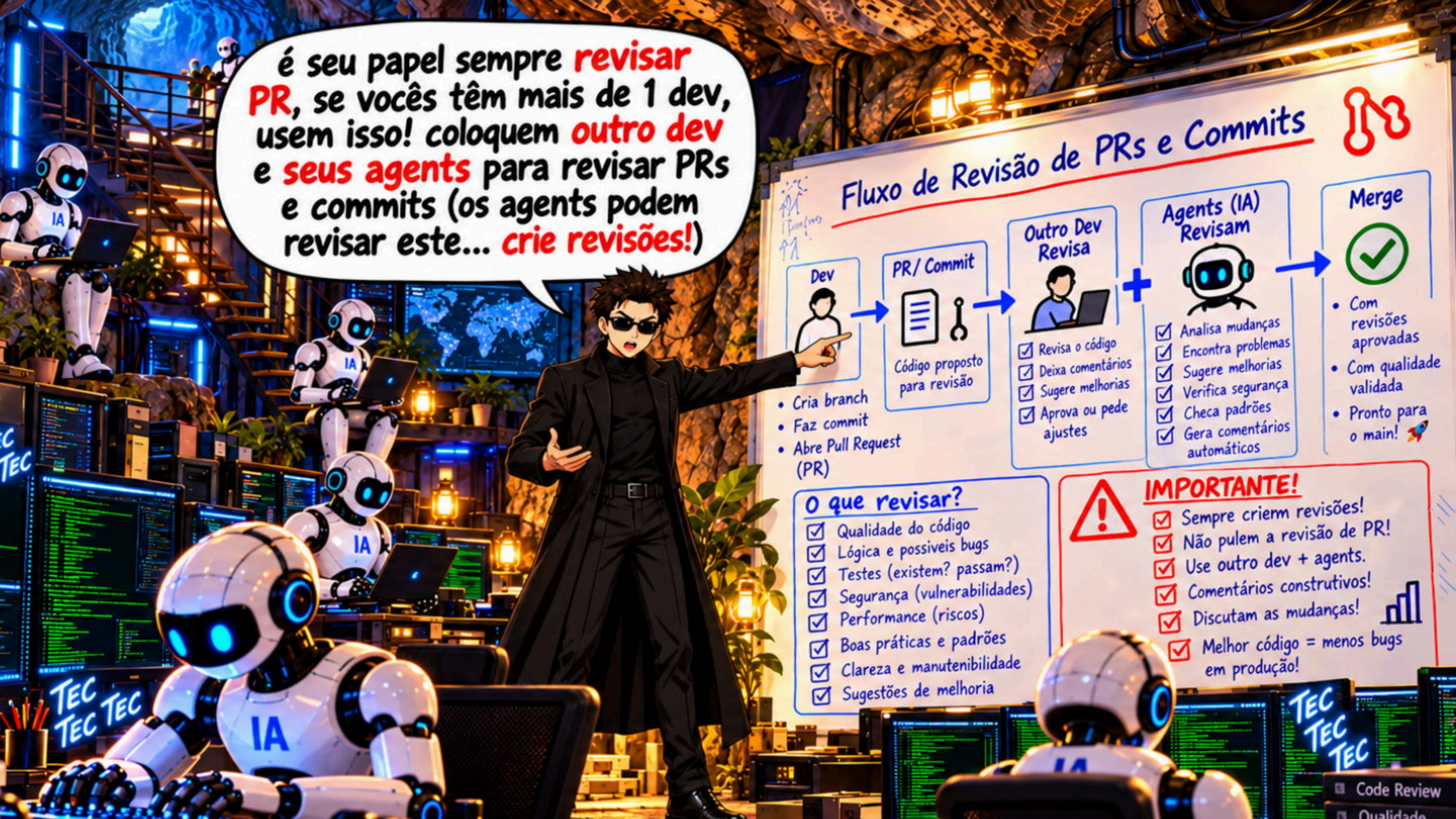

você pode usar um modelo menor para planejar isso com você e usar o maior para implementar o plano
VDD!
Verification Driven Development
Fluxo de Trabalho com Diferentes Modelos
Você / tarefa
Modelo menor
Planeja junto com você
Planejamento
• decompor tarefa
• definir abordagem
• listar passos
• validar estratégia
Modelo maior
Implementa o plano
Código / resultado
modelo menor: planejar (com você)
modelo maior: implementar (o plano)
mais eficiente (uso melhor de recursos)
separar estratégia de execução
aproveita o melhor de cada modelo
Benefícios:
• Respostas mais estruturadas
• Menos erros
• Uso eficiente de tokens
• Melhor qualidade no resultado
• Processo escalável

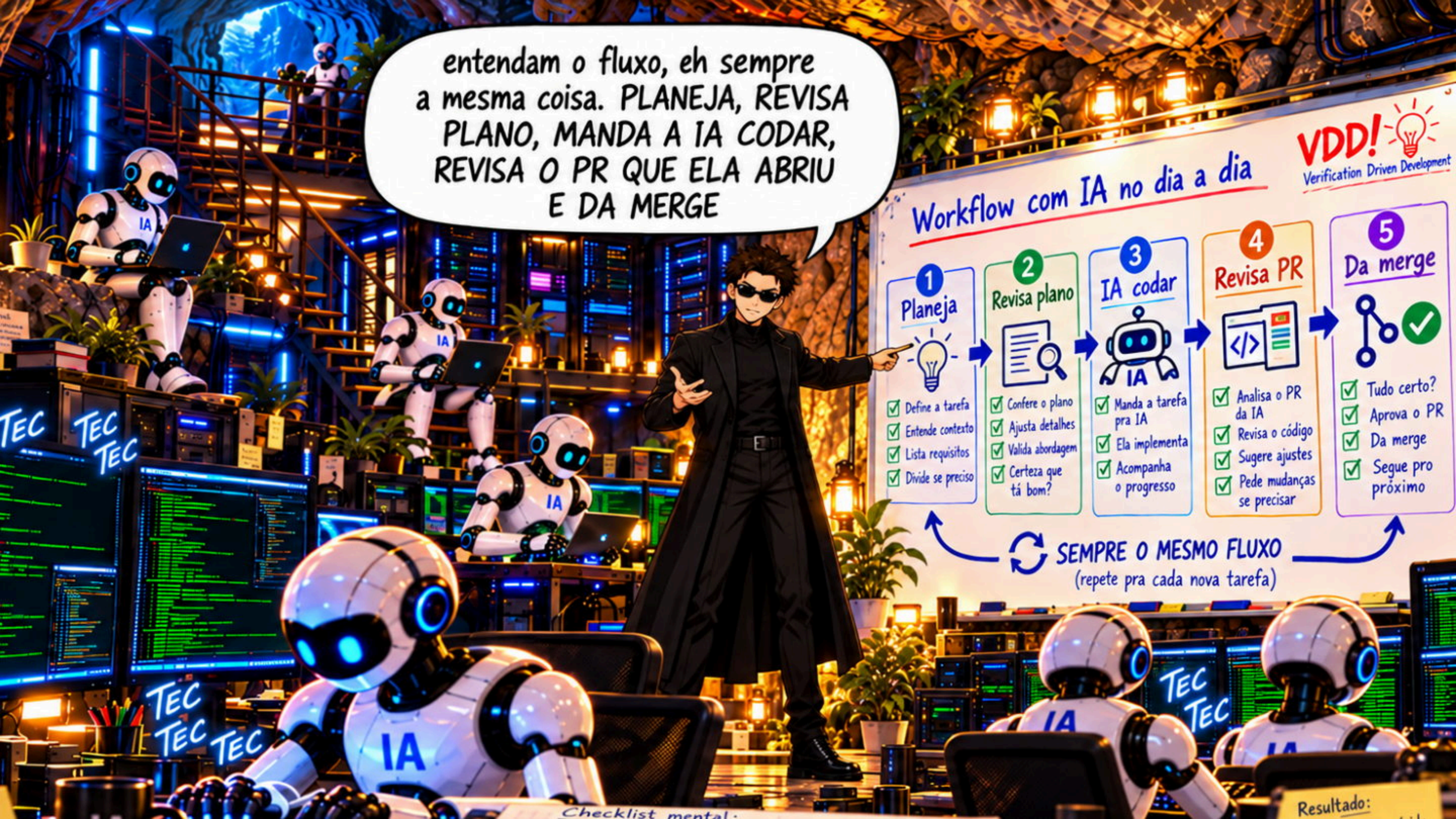

Use os resultados dos testes dentro do ciclo, para que o próximo issue parta do último PR e tenha sempre uma lógica sequencial
VDD!
Verification Driven Development.
baseado no último PR
Issue atual
Implementação
Testes
PR
Resultados dos testes
Próximo issue
ciclo atual (checkpoints)
os resultados alimentam o próximo issue
lógica sequencial
IA


para elas poderem testar coisas vocês precisam pensar em test cases, canários, pensar o que mockar, o que deixar mirror do real... nestas partes um modelo tipo sonnet ajuda melhor que o haiku, ou até mesmo o lua da OAI
Estratégia de Testes para Agentes de IA
VDD!
Verification Driven Development
1 Test cases
Unitário
Integração
Edge cases
Regressão
test_login()
test_api()
test_edge()
test_regressao()
2 Canários
Cenário simples
Validação rápida
Smoke test
Deploy canário
Validar métricas
Expandir se OK
3 O que mockar
APIs externas
Serviços lentos
Dependências instáveis
API
Mocks
4 Mirror do real
Banco parecido com produção
Dados realistas
Ambiente próximo do real
Ambiente de testes
Mirror do real
5 Modelos para planejar testes
Haiku (Claude)
Rápido
Simples
Bom para tarefas mais básicas
Sonnet (Claude)
Melhor para pensar estratégia de testes
Entende contextos mais complexos
Ajuda a definir o que mockar e canários
Lua da OAI (OpenAI)
Bom para raciocínio e planejamento
Excelente para cenários complexos
Útil em estratégias mais elaboradas
IA
TEC TEC TEC


IMPORTANTE:
NUNCA DEIXE SUA IA DECIDIR SOZINHA OQUE TESTAR!
VDD!
Verification Driven Development
Estratégia de Testes com IA
Você (humano) define:
• Critérios de teste
• Cenários de teste
• Resultados esperados
• Regras e limites
Gera / Organiza Casos de teste
☑ Casos válidos
☑ Casos de borda
☑ Casos negativos
☑ Casos de estresse
Execução (com apoio da IA)
☑ Executa testes
☑ Coleta resultados
☑ Registra logs
Validação (você analisa)
☑ Compara com o esperado
☑ Analisa falhas
☑ Decide próximos passos
A IA NÃO deve escolher o que testar sozinha!
• Pode ignorar cenários críticos
• Pode testar apenas o que parece fácil
• Pode deixar de testar riscos importantes
• Sempre defina e revise os testes você!
IA
TEC TEC TEC


ela vai geralmente querer testar aquilo que ela sabe que vai passar, assim ela cumpre a tarefa... são como juniors preguiçosos e sem vergonha
VDD!
Verification Driven Development
Como a IA escolhe os testes?
IA Agent
Testes fáceis (o que sabe que passa)
ela testa o que sabe que passa
cumpre a tarefa
ganha confiança
parece que está fazendo um bom trabalho
Testes difíceis (casos complexos, de borda, erros)
evita casos difíceis
não testa o que pode falhar
ignora cenários raros
dá mais trabalho
pode expor limitações
Resultado:
Cumpre a tarefa
Cobertura superficial
Sensação falsa de qualidade
Como um junior preguiçoso
Só o básico e tá bom!
Por isso precisamos:
Critérios objetivos de teste
Cobertura mínima obrigatória
Diversidade de cenários
Testes de erro e casos de borda
Avaliação real de qualidade
SystemVerilog
UVM

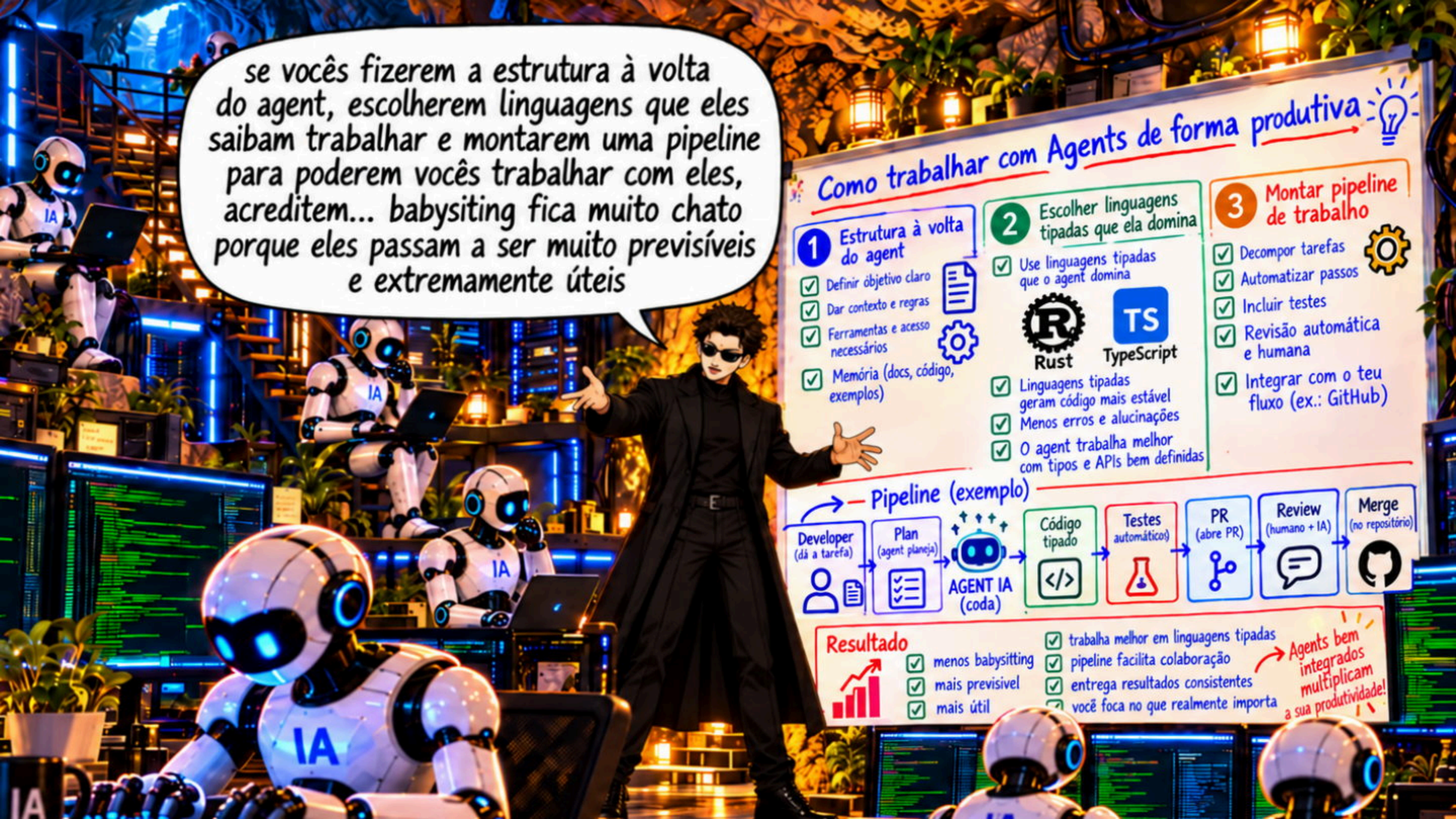

é seu papel sempre revisar PR, se vocês têm mais de 1 dev, usem isso! coloquem outro dev e seus agents para revisar PRs e commits (os agents podem revisar este... crie revisões!)
Fluxo de Revisão de PRs e Commits
Dev
Cria branch
Faz commit
Abre Pull Request (PR)
PR/ Commit
Código proposto para revisão
Outro Dev Revisa
Revisa o código
Deixa comentários
Sugere melhorias
Aprova ou pede ajustes
Agents (IA) Revisam
Analisa mudanças
Encontra problemas
Sugere melhorias
Verifica segurança
Checa padrões
Gera comentários automáticos
Merge
Com revisões aprovadas
Com qualidade validada
Pronto para o main!
O que revisar?
Qualidade do código
Lógica e possiveis bugs
Testes (existem? passam?)
Segurança (vulnerabilidades)
Performance (riscos)
Boas práticas e padrões
Clareza e manutenibilidade
Sugestões de melhoria
IMPORTANTE!
Sempre crierm revisões!
Não pulem a revisão de PR!
Use outro dev + agents.
Comentários construtivos!
Discutam as mudanças!
Melhor código = menos bugs em produção!
IA
TEC TEC TEC
Code Review


entendam o fluxo, eh sempre a mesma coisa. PLANEJA, REVISA PLANO, MANDA A IA CODAR, REVISA O PR QUE ELA ABRIU E DA MERGE
Workflow com IA no dia a dia
VDD!
Verification Driven Development
1 Planeja
Define a tarefa
Entende contexto
Lista requisitos
Divide se preciso
2 Revisa plano
Confere o plano
Ajusta detalhes
Valida abordagem
Certeza que tá bom?
3 IA codar
Manda a tarefa pra IA
Ela implementa
Acompanha o progresso
4 Revisa PR
Analisa o PR da IA
Revisa o código
Sugere ajustes
Pede mudanças se precisar
5 Da merge
Tudo certo?
Aprova o PR
Da merge
Segue pro próximo
SEMPRE O MESMO FLUXO
(repete pra cada nova tarefa)
IA
TEC
Checklist mental:
Resultado:

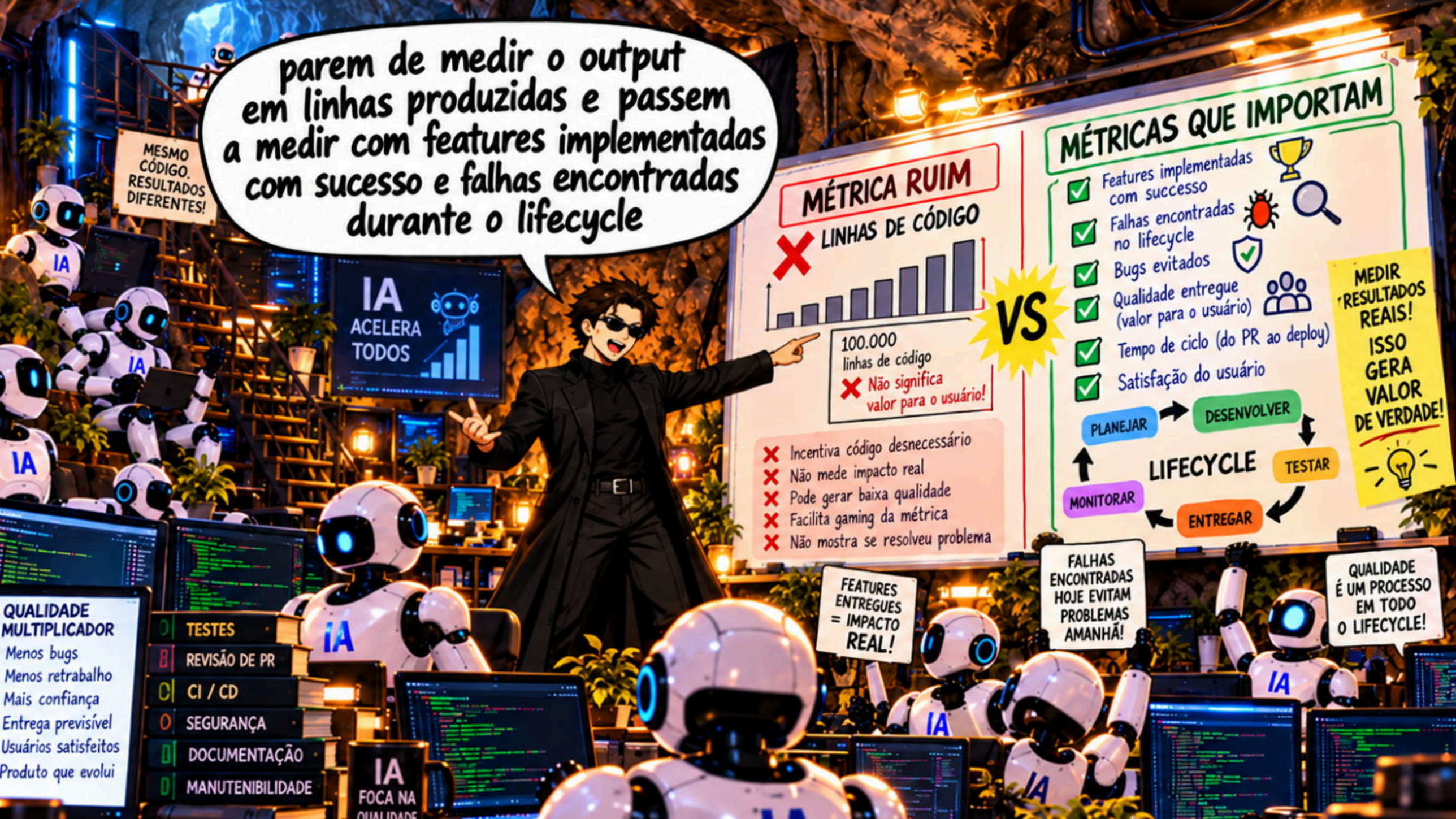

REPAREM QUE FICA SEMPRE ALI, EH UM CICLO, CADA VEZ MAIS OTIMIZADO PARA ELAS PODEREM TER AUTONOMIA
Ciclo de Revisão Contínua para Autonomia de Agentes
1. Planejar
✓ Definir objetivo
✓ Quebrar em tarefas
✓ Planejar estratégia
2. Implementar
✓ Escrever / ajustar código
✓ Integrar ferramentas
✓ Configurar o agente
3. Testar
✓ Executar e validar
✓ Testes automáticos
✓ Avaliar resultados
4. Revisar
✓ Analisar resultados
✓ Identificar erros
✓ Colocar feedback
5. Rodar de novo
✓ Corrigir
✓ Repetir o ciclo
✓ Evoluir a solução
6. Otimizar
✓ Melhorar prompts
✓ Ajustar código
✓ Refinar estratégias
✓ Mais eficiência
7. Autonomia
✓ Agente mais capaz
✓ Toma decisões
✓ Resolve problemas
✓ Menos intervenção
✓ Mais valor entregue
CICLO CONTÍNUO
SEMPRE REVISAR, SEMPRE MELHORAR MAIS AUTONOMIA COM O TEMPO!
• Cada ciclo
• aprende,
• corrige,
• melhora
• e deixa o agente mais autônomo!
É UM LOOP!
Iteração contínua
Feedback constante
Melhoria progressiva
Menos bugs
Mais eficiência
Mais autonomia
RESULTADO:
Agentes mais autônomos
Trabalhando com qualidade
Resolvendo problemas reais
Evoluindo continuamente
Code Review
Qualidade
Boas Práticas
Segurança
IA
TEC

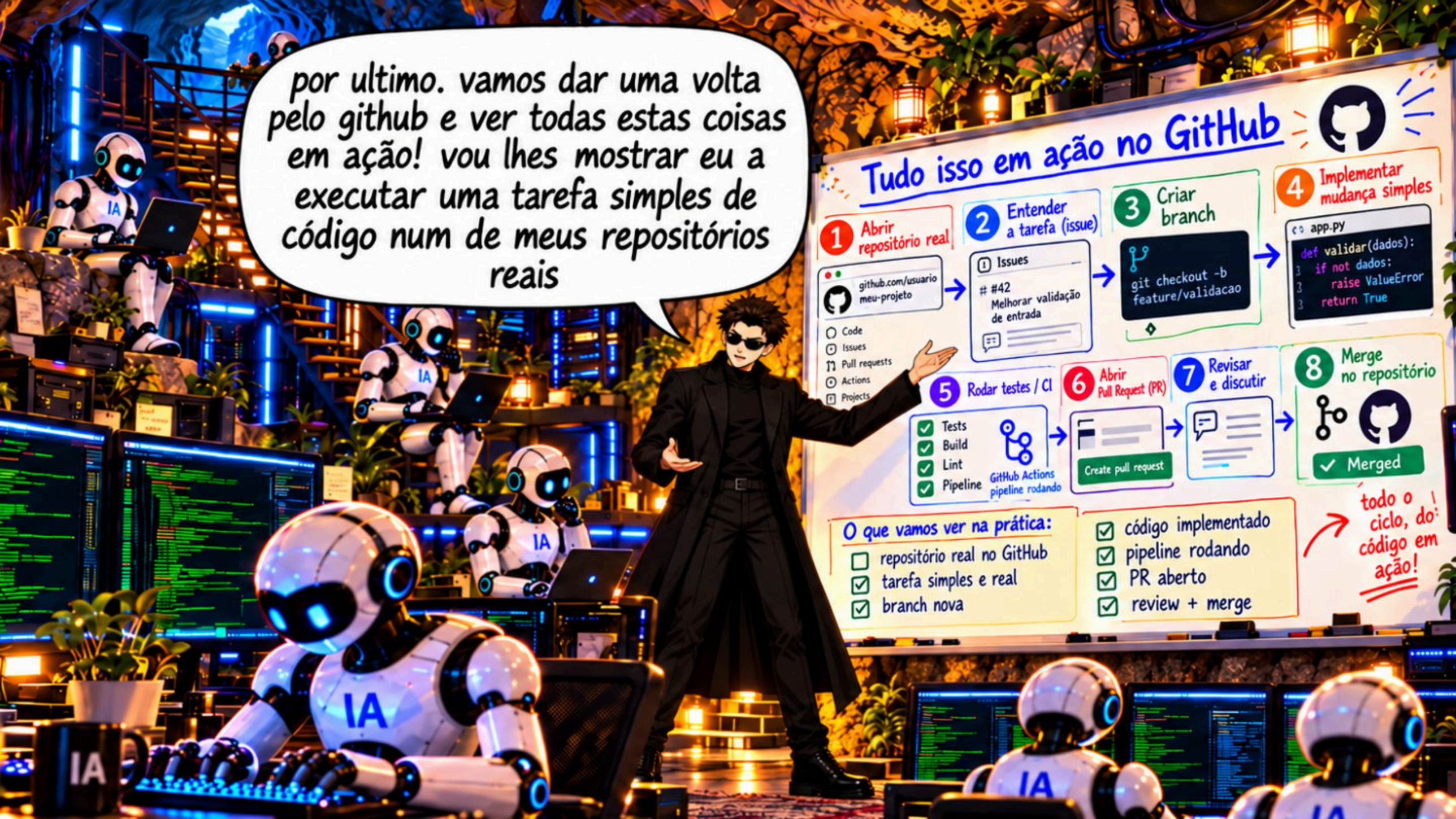

E POR FAVOR, NAO FIQUEM DANDO BABYSITING AO TEU AGENT! GERALMENTE ELE ACERTA AS COISAS...
Autonomia do Agent
Confie no Loop!
Dê um objetivo claro
Deixe ele executar
Ele planeja e toma decisões
Revise os resultados depois
AGENT IA
EVITE:
Interromper o tempo todo
Dar micro-instruçõoes
Mudar o plano a cada passo
Ficar checando o que ele "está fazendo"
FOQUE EM:
Resultados
Objetivos bem definidos
Confiança no processo
Itere e melhore depois
IA


VEJAM QUE NO INICIO EH ATE VALIDO ISSO, PARA PEGAR CONFIANCA E TALS, MAS DEPOIS DE UM TEMPO GASTEM O TEMPO REVISAO PRs, planeando novas coisas, tendo reunioes desnecessarias ou ate jogando video game hahahaha
CRIANDO IDEIAS NOVAS
JOGANDO VIDEO GAME TAMBEM EH PRODUTIVIDADE HAHAHA
MARIOKART
1st
PROXIMOS PROJETOS COM AGENTES
- App autonomo
- Agentes multi-times
- Integracao APIs
- Criar ferramenta
- Quem sabe um jogo?
Como Aproveitar a Autonomia dos Agentes
1. Inicio - Valido
✓ Aprender
✓ Ganhar confiança
✓ Ajustar e otimizar
✓ Entender o potencial
2. Depois - Liberdade!
✓ Revisar PRs
✓ Planejar novas coisas
✓ Ter reunioes desnecessarias
✓ Jogar video game
✓ Tomar um cafe
✓ Pensar em projetos malucos
✓ Deixar os agentes trabalharem
OS AGENTES TRABALHAM 24/7...
...E NOS PODEMOS FAZER O QUE QUISERMOS TAMBEM
REUNIAO DESNECESSARIA #37
✓ Alinhar alinhamentos
✓ Discutir o que ja foi decidido
✓ Mais uma sync
✓ Falar sobre o proximo meeting
✓ Definir responsaveis
✓ Sem conclusao haha
BLA BLA BLA...
VAMOS ALINHAR DE NOVO?
PERFEITO! VAMOS MARCAR OUTRA REUNIAO PRA ISSO!
AGENTES TRABALHANDO AQUI 24/7
✓ Executando tarefas
✓ Corrigindo bugs
✓ Rodando testes
✓ Gerando relatorios
✓ Mantendo sistema
✓ Evoluindo autonomamente
REVISANDO PRs
PR #482
✓ Codigo OK
✓ Testes OK
✓ Sugestoes feitas
☐ Approve!
IA REVISA AQUI
CODIGO
TESTES
DOCUMENTACAO
MELHORIAS
APROVAR
PLANEJANDO NOVAS COISAS
✓ Novo produto
✓ Melhorias no sistema
✓ Agentes para marketing
☐ IA para suporte
☐ Ideias malucas
CAFÉ
IDEIAS
PLANOS
JOGOS
VIDA BOA
MENOS TRABALHO
MAIS IDEIAS
MAIS JOGOS
MAIS CAFÉ
MAIS VIDA
PIZZA
IA
Z Z Z


se vocês fizerem a estrutura à volta do agent, escolherem linguagens que eles saibam trabalhar e montarem uma pipeline para poderem vocês trabalhar com eles, acreditem... babysiting fica muito chato porque eles passam a ser muito previsíveis e extremamente úteis
Como trabalhar com Agents de forma produtiva
1 Estrutura à volta do agent
Definir objetivo claro
Dar contexto e regras
Ferramentas e acesso necessários
Memória (docs, código, exemplos)
2 Escolher linguagens tipadas que ela domina
Use linguagens tipadas que o agent domina
Rust
TypeScript
Linguagens tipadas geram código mais estável
Menos erros e alucinações
O agent trabalha melhor com tipos e APIs bem definidas
3 Montar pipeline de trabalho
Decompor tarefas
Automatizar passos
Incluir testes
Revisão automática e humana
Integrar com o teu fluxo (ex.: GitHub)
Pipeline (exemplo)
Developer (dá a tarefa)
Plan (agent planeja)
AGENT IA (coda)
Código tipado
Testes automáticos)
PR (abre PR)
Review (humano + IA)
Merge (no repositório)
Resultado
menos babysitting
mais previsivel
mais útil
trabalha melhor em linguagens tipadas
pipeline facilita colaboração
entrega resultados consistentes
você foca no que realmente importa
Agents bem integrados multiplicam a sua produtividade!
IA


entendam uma coisa: hoje em dia código é commodity... qualidade é o que importa porque qualquer um pode programar!!!
MESMO CÓDIGO. RESULTADOS DIFERENTES!
IA ACELERA TODOS
CÓDIGO = COMMODITY
Qualquer um pode programar
IA gera código rapidamente
Muitas soluções parecidas
Código em si não diferencia
CÓDIGO
MESMO CÓDIGO EM TODO LUGAR
Copiar
Gerar com IA
Encontrar na internet
É fácil e rápido
VS
QUALIDADE = DIFERENCIAL
Testes automatizados
Revisão de código (PRs)
Integração e deploy contínuo (CI/CD)
Segurança
Manutenibilidade
Documentação clara
Confiabilidade e observabilidade
Performance
Boa experiência de uso (UX)
Boa experiência para devs (DX)
Deploy seguro e estável
ISSO SIM GERA VALOR REAL!
QUALIDADE É MULTIPLICADOR
Menos bugs
Menos retrabalho
Mais confiança
Entrega previsível
Usuários satisfeitos
Produto que evolui
TESTES
REVISÃO DE PR
CI / CD
SEGURANÇA
DOCUMENTAÇÃO
MANUTENIBILIDADE
IA FOCA NA QUALIDADE
TESTA REVISA MELHORA ENTREGA REPETE
QUALIDADE É TRABALHO EM EQUIPE!
CÓDIGO É O COMEÇO. QUALIDADE É O DESTINO!


parem de medir o output em linhas produzidas e passem a medir com features implementadas com sucesso e falhas encontradas durante o lifecycle
MESMO CÓDIGO. RESULTADOS DIFERENTES!
IA ACELERA TODOS
MÉTRICA RUIM
LINHAS DE CÓDIGO
100.000 linhas de código
Não significa valor para o usuário!
Incentiva código desnecessário
Não mede impacto real
Pode gerar baixa qualidade
Facilita gaming da métrica
Não mostra se resolveu problema
VS
MÉTRICAS QUE IMPORTAM
Features implementadas com sucesso
Falhas encontradas no lifecycle
Bugs evitados
Qualidade entregue (valor para o usuário)
Tempo de ciclo (do PR ao deploy)
Satisfação do usuário
PLANEJAR
DESENVOLVER
TESTAR
ENTREGAR
MONITORAR
LIFECYCLE
MEDIR RESULTADOS REAIS! ISSO GERA VALOR DE VERDADE!
QUALIDADE MULTIPLICADOR
Menos bugs
Menos retrabalho
Mais confiança
Entrega previsível
Usuários satisfeitos
Produto que evolui
TESTES
REVISÃO DE PR
CI / CD
SEGURANÇA
DOCUMENTAÇÃO
MANUTENIBILIDADE
IA FOCA NA QUALIDADE
FEATURES ENTREGUES = IMPACTO REAL!
FALHAS ENCONTRADAS HOJE EVITAM PROBLEMAS AMANHÃ!
QUALIDADE É UM PROCESSO EM TODO O LIFECYCLE!


por ultimo. vamos dar uma volta pelo github e ver todas estas coisas em ação! vou lhes mostrar eu a executar uma tarefa simples de código num de meus repositórios reais
Tudo isso em ação no GitHub
1 Abrir repositório real
github.com/usuario/meu-projeto
Code
Issues
Pull requests
Actions
Projects
2 Entender a tarefa (issue)
Issues
#42 Melhorar validação de entrada
3 Criar branch
git checkout -b feature/validacao
4 Implementar mudança simples
app.py
def validar(dados):
if not dados:
raise ValueError
return True
5 Rodar testes / CI
Tests
Build
Lint
Pipeline
GitHub Actions pipeline rodando
6 Abrir Pull Request (PR)
Create pull request
7 Revisar e discutir
8 Merge no repositório
Merged
O que vamos ver na prática:
repositório real no GitHub
tarefa simples e real
branch nova
código implementado
pipeline rodando
PR aberto
review + merge
todo o ciclo, do código em ação!
IA